

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos

Los mini-ciclos se definieron para ciclo 1 siguiendo el orden de dependencias entre clases: primero las figuras extendidas del paquete shapes (SymbolCircle, SymbolRectangle, SymbolTriangle y la interfaz SymbolShape), luego Symbol, después Wheel, y finalmente SlotMachine porque cada clase depende funcionalmente de la anterior, lo que permitió compilar y probar cada clase mientras se creaba.

Los mini-ciclos que se definieron para ciclo 2 fueron primero el swap(wheel1, wheel2) y lock/unlock debido a que ambos requieren el mismo tipo de cambio en Wheel (nuevo atributo locked) y en SlotMachine, por lo que se agruparon en un mismo mini-ciclo después spin(wheel, steps) y spin(setSymbols) debido a que ambos extienden el comportamiento de giro ya existente, y comparten un mismo patrón al final siendo isJackpot() + cambio de color + refresco visual y por ultimo las pruebas de unidad (SlotMachineC2Test), pruebas compartidas (SlotMachineCC2Test) y pruebas de aceptación porque se dejaron como mini-ciclo final, una vez estabilizada la implementación, para evitar reescribir pruebas por cambios de diseño a mitad de camino

Para el ciclo 3 primero se hizo el nuevo constructor ya que en eso se basaban los otros métodos que se debían implementar, luego se creo un nuevo método distinctCurrentSymbols() que retorna la cantidad de símbolos diferentes que se muestran, con eso van a trabajar los métodos solve y simulate, al tener eso se creó la clase SlotMachineContest y se creo el método solve(n) primero ya que este sigue la lógica que se debe tener en ambos, luego se usó el mismo código de ese método en simulate(n).

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

El ciclo 1 actualmente cumple con todos los requisitos, se completaron todos los miniciclos definidos

Los mini-ciclos del ciclo 2 están completados los 4 requisitos funcionales nuevos están implementados y verificados con diagramas de secuencia actualizados se cuenta con 8 pruebas de unidad propias, 2 pruebas compartidas, y 2 pruebas de aceptación documentadas

Se completaron todos los mini-ciclos y se implementaron todos los requisitos requeridos en el pdf, se actualizo el diagrama de clases añadiendo la nueva clase y sus métodos y se crearon los diagramas de secuencia de los nuevos métodos.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Díaz: C1: 18 horas	C2: 18 horas	C3: 18 horas	total: 54 horas
Rojas: C1: 18 horas	C2: 15 horas	C3: 19 horas	total: 52 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

En el ciclo 1 lograr que la máquina reutilizara el paquete shapes tal como lo pedía el enunciado, sin tener que modificarlo, gracias al uso de una interfaz que conecta la lógica del juego con la figura que se dibuja en pantalla.

En el ciclo 2 lograr que Symbol delegara completamente la creación y el comportamiento de la figura gráfica a través de la interfaz SymbolShape, incluyendo el método de fábrica createShape(shapeType), sin que Wheel ni SlotMachine necesitaran conocer nunca las clases concretas (SymbolCircle, SymbolRectangle, SymbolTriangle) permitiendo que agregar el parámetro shapeType al flujo de creación de símbolos solo requiriera tocar Symbol y Wheel, sin cascada de cambios hacia SlotMachine

En el ciclo 3 el mayor logro fue entender como funcionaba solve correctamente y poder implementar bien la lógica en el código, algo que costó bastante tiempo de razonamiento por parte de ambos.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El en ciclo 1 mayor problema técnico fue mantener la máquina visualmente centrada mientras su ancho cambiaba al agregar o eliminar ruedas. Se resolvió llevando manualmente atributos de posición en cada clase que lo requería, y aplicando la fórmula de desplazamiento (distancia = posición destino

– posición actual) junto con los métodos `moveHorizontal`/`moveVertical` cada vez que el tamaño o la posición debían actualizarse.

En el ciclo 2 la validación de `isVisible` estaba fallando en varios métodos, la razón era que el método `setVisible` nunca actualizaba `isVisible` a `true` y que las tres figuras originales (`Circle`, `Rectangle`, `Triangle`) interpretan su coordenada `xPosition` de forma distinta: `Triangle` la trata como el centro horizontal, mientras `Rectangle` y `Circle` la tratan como la esquina izquierda.

El mayor desafío técnico del Ciclo 3 fue igualar las ruedas a ciegas usando solo `distinctSymbols()`, sin conocer sus símbolos individuales. Lo solucionamos moviendo el grupo igual y comparando la cantidad de diferentes antes y después: la única posición donde la cantidad aumentaba revelaba el punto exacto de coincidencia para alinear la rueda.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

En el ciclo 1 como equipo, dividimos el trabajo entre los dos para optimizar el tiempo de desarrollo. Como compromiso de mejora, nos proponemos trabajar más en conjunto para mejorar la coherencia del proyecto.

En el ciclo 2 como equipo nos dividimos el trabajo para poder optimizar tiempo de desarrollo esta vez trabajando más en conjunto y explicándonos el uno al otro los cambios que hacíamos y las cosas que uno sabía pero el otro no, como compromiso de mejora, nos enfocaremos más en la investigación previa y revisiones largas antes de cada entrega para revisar que no hayan bugs ni errores que no sabíamos que existían.

En el ciclo 3 igual que en los anteriores ciclos, cada que se realizaba un cambio al proyecto se buscaba explicar de la mejor manera al otro lo que se hizo para así ambos tener el mayor conocimiento posible acerca del proyecto.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

En el ciclo 1 consideramos que `Collective Code Ownership` fue la práctica más útil, ya que nos permitió dividir el trabajo y hacer cambios cuando quisieramos.

En el ciclo 2 consideramos que `pair programming` fue la que más nos sirvió ya que nos ayudaba a no saturarnos de código y que de paso ambos fuéramos aprendiendo aún más como usar Java.

En el ciclo 3 `pair programming` fue de bastante ayuda para poder trabajar de una manera mucho más cómoda entre los dos.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Las referencias que más usamos fueron el proyecto de `shapes` y la documentación de Java, siendo esta la más útil ya que nos indicaba como debíamos implementar el código.

Oracle Corporation. Overview(Java SE 21 & JDK 21). Oracle.

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html>

En el ciclo 3 la referencia más útil fue la solución del problema de la maratón ya que nos ayudó a entender como debía funcionar correctamente el solve.